13. Recap: ggplot2 and data transformation

Principles of Data Science with R

Dr. Uma Ravat

PSTAT 10

What we did in the case study

Import the data, **tidy** the column names, **transform** it, **visualize** it, and **communicate** what we found.

Today we name the pieces you already used, so you can reuse them on any data set.

- Part 1: the structure of a ggplot
- Part 2: the data transformation verbs
- Part 3: tidy data and pivoting (quick here, more later again)

Part 1: Building plots with ggplot2

Every ggplot has the same skeleton

```
ggplot(data = DATA, aes(x = X-VAR, y = Y-VAR, ...)) +
  GEOM_FUNCTION() +
  ...
```

- **data**: the data frame
- **aesthetics** (`aes()`): which variables map to which visual properties (`x`, `y`, `color`, `fill`, `size`, ...)
- **geoms**: how the data are drawn (points, bars, boxes, lines, ...)
- layers are added with +

Aesthetics: mapping versus setting

```
ggplot(drivers_clean, aes(x = pct_alcohol, y = fatal_rate)) +
  geom_point(color = "firebrick", size = 3, alpha = 0.7)
```



- `color = "firebrick"` is **set** outside `aes()`: every point looks the same.
- `aes(color = pct_speeding)` would **map** a variable inside `aes()`: the color depends on the data.

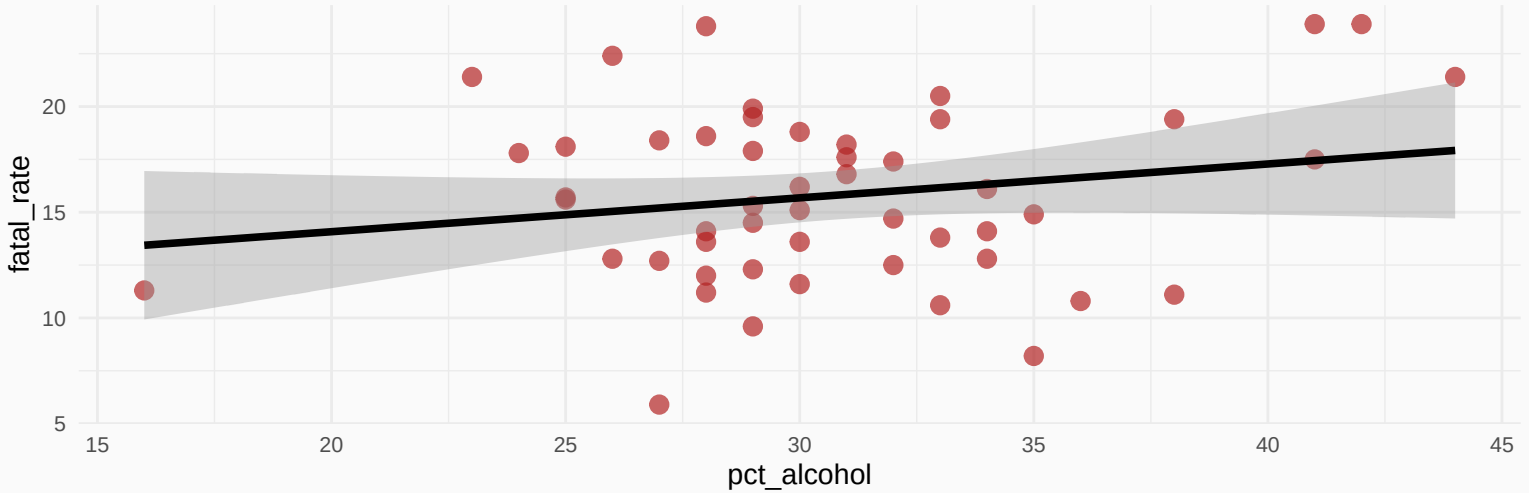
Your turn: mapping or setting?

For each, say whether the point color is *mapped* or *set*, and describe the plot.

```
geom_point(aes(color = pct_speeding))
geom_point(color = "darkgreen")
geom_point(aes(color = "darkgreen"))
```

Layers: points plus a trend line

```
ggplot(drivers_clean, aes(x = pct_alcohol, y = fatal_rate)) +
  geom_point(color = "firebrick", size = 2, alpha = 0.7) +
  geom_smooth(method = "lm", se = TRUE, color = "black")
```



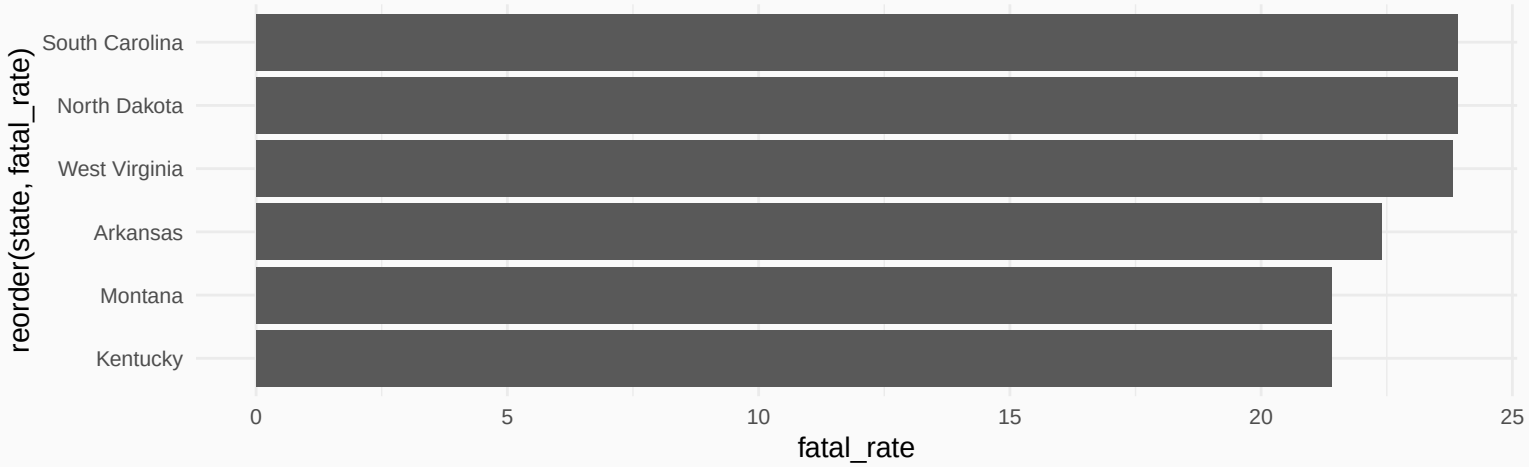
- `geom_smooth(method = "lm")` adds a fitted line, and `se = TRUE` shows the uncertainty band.
- Aesthetics in `ggplot()` are shared by all layers. Aesthetics inside a geom apply to that layer only.

Choosing a geom

Question	Geom
How is one numeric variable spread?	`geom_histogram()`
Compare a numeric variable by group?	`geom_boxplot()`
Relationship of two numeric variables?	`geom_point()`
Trend through scattered points?	`geom_smooth()`
Counts of categories?	`geom_bar()`
Bar heights already computed?	`geom_col()`

geom_col() versus geom_bar()

```
drivers_clean |>
  slice_max(fatal_rate, n = 5) |>
  ggplot(aes(x = fatal_rate, y = reorder(state, fatal_rate))) +
  geom_col()
```

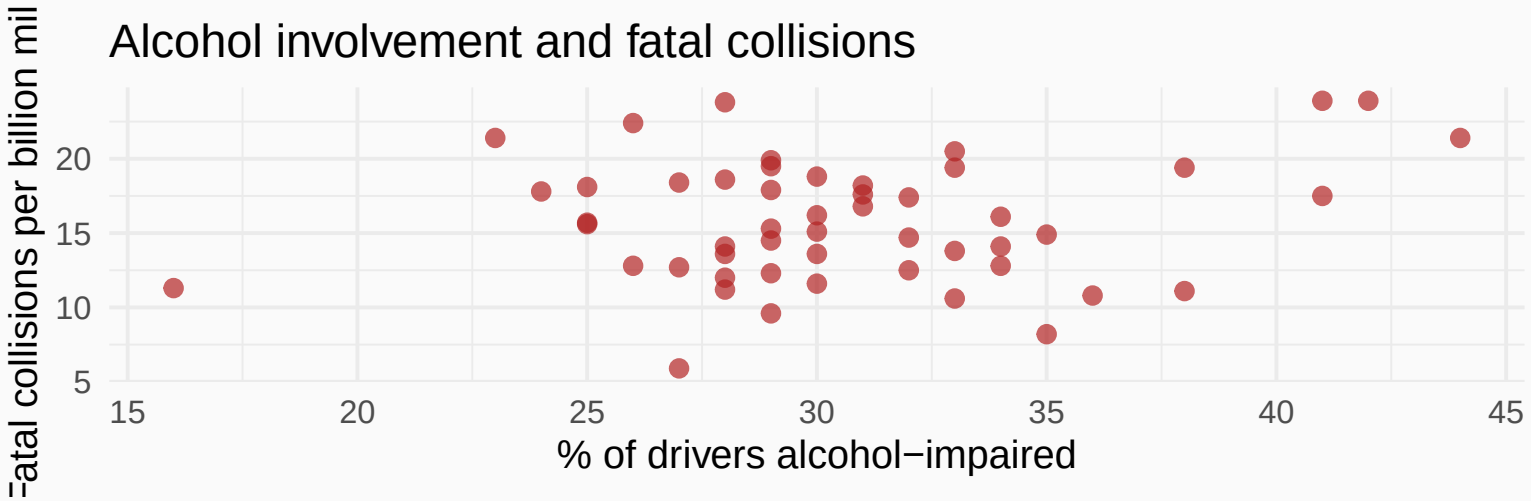


`geom_col()` uses the heights you give it (here `fatal_rate`).

`geom_bar()` counts rows for you. Our first bar chart of every state was hard to read, so we limited it to five states and ordered the bars.

Polish: labels, color scales, themes

```
ggplot(drivers_clean, aes(x = pct_alcohol, y = fatal_rate)) +
  geom_point(color = "firebrick", size = 2, alpha = 0.7) +
  labs(title = "Alcohol involvement and fatal collisions",
       x = "% of drivers alcohol-impaired",
       y = "Fatal collisions per billion miles") +
  theme_minimal()
```



A plot without clear labels is a picture; a plot with a title, labels, and units is an argument.

Your turn: fill in the blanks

Make a histogram of insurance premiums.

```
drivers_clean |>
  ggplot(aes(x = _______)) +
  geom_______(bins = 12) +
  labs(x = "_______", y = "_______") +
  theme_minimal()
```

Part 2: Transforming data

The pipe: read it as "and then"

```
drivers_clean |>
  select(state, fatal_rate) |>
  arrange(desc(fatal_rate)) |>
  head(3)
```

```
## # A tibble: 3 x 2
##   state          fatal_rate
##   <chr>               <dbl>
## 1 North Dakota         23.9
## 2 South Carolina       23.9
## 3 West Virginia        23.8
```

- Start with `drivers_clean`, *and then* select two columns, *and then* sort, *and then* keep the top three.
- The pipe passes the result on the left in as the first argument of the function on the right.

The core verbs

Verb	What it does
`rename()`	change column names
`select()`	choose columns
`filter()`	choose rows that meet a condition
`arrange()`	sort rows
`mutate()`	create or change columns
`group_by()` + `summarize()`	summarize each group
`slice_max()`, `slice_min()`	keep the rows with the largest or smallest values
`count()`	count rows per group

`mutate()` creates columns

```
drivers_clean |>
  mutate(pct_distracted = 100 - pct_not_distracted) |>
  select(state, pct_not_distracted, pct_distracted) |>
  head(3)
```

```
## # A tibble: 3 x 3
##   state   pct_not_distracted pct_distracted
##   <chr>                <dbl>          <dbl>
## 1 Alabama                 96              4
## 2 Alaska                  90             10
## 3 Arizona                 84             16
```

The data had a column for drivers who were *not* distracted. We created the more natural column, `pct_distracted`.

case_when() recodes values

```
drivers_regions <- drivers_clean |>
  mutate(
    region = case_when(
      state %in% c("Maine", "Vermont")  ~ "Northeast",
      state %in% c("Ohio", "Iowa")      ~ "Midwest",
      state %in% c("Texas", "Georgia")  ~ "South",
      .default = "Other"
    )
  )
count(drivers_regions, region)
```

```
## # A tibble: 4 x 2
##   region        n
##   <chr>     <int>
## 1 Midwest       2
## 2 Northeast     2
## 3 Other        45
## 4 South         2
```

Conditions are checked **in order**, the first `TRUE` wins, and `.default` catches everything else. In the case study we listed every state, so we did not need a default. If we had forgotten one, its region would have been `NA`.

Always check your recode

```
drivers_clean |>
  mutate(
    region = case_when(
      state %in% c("Maine", "Vermont")  ~ "Northeast",
      state %in% c("Ohio", "Iowa")      ~ "Midwest"
    )
  ) |>
  count(region)
```

```
## # A tibble: 3 x 2
##   region        n
##   <chr>     <int>
## 1 Midwest       2
## 2 Northeast     2
## 3 <NA>         47
```

Without a `.default`, any state that matches no condition becomes `NA`. Counting the result is a quick way to catch this.

group_by() and summarize()

```
drivers_regions |>
  group_by(region) |>
  summarize(
    n_states = n(),
    avg_fatal_rate = mean(fatal_rate),
    avg_premium = mean(insurance_premium)
  ) |>
  arrange(desc(avg_fatal_rate))
```

```
## # A tibble: 4 x 4
##   region    n_states avg_fatal_rate avg_premium
##   <chr>        <int>          <dbl>       <dbl>
## 1 South            2           17.5        959.
## 2 Other           45           15.8        902.
## 3 Midwest          2           14.9        673.
## 4 Northeast        2           14.4        689.
```

group_by() splits the data, summarize() collapses each group to a single row.

Your turn: put the steps in order

Find the five states with the highest insurance premiums, showing only the state and the premium.

Put these steps in the right order.

- `select(state, insurance_premium)`
- `slice_max(insurance_premium, n = 5)`
- `drivers_clean |>`

Part 3: Tidy data

Tidy data

- Each **variable** is a column.
- Each **observation** is a row.
- Each type of observational unit is a table.

```
head(drivers_clean, 3)
```

```
## # A tibble: 3 x 8
##   state fatal_rate pct_speeding pct_alcohol pct_not_distracted
##   <chr>      <dbl>        <dbl>       <dbl>               <dbl>
## 1 Alab~       18.8           39          30                  96
## 2 Alas~       18.1           41          25                  90
## 3 Ariz~       18.6           35          28                  84
## # i 3 more variables: pct_no_previous <dbl>,
## #   insurance_premium <dbl>, losses <dbl>
```

To compare three causes of fatal crashes (speeding, alcohol, distraction) with one plot, we needed cause to be a column.

pivot_longer() made cause a column

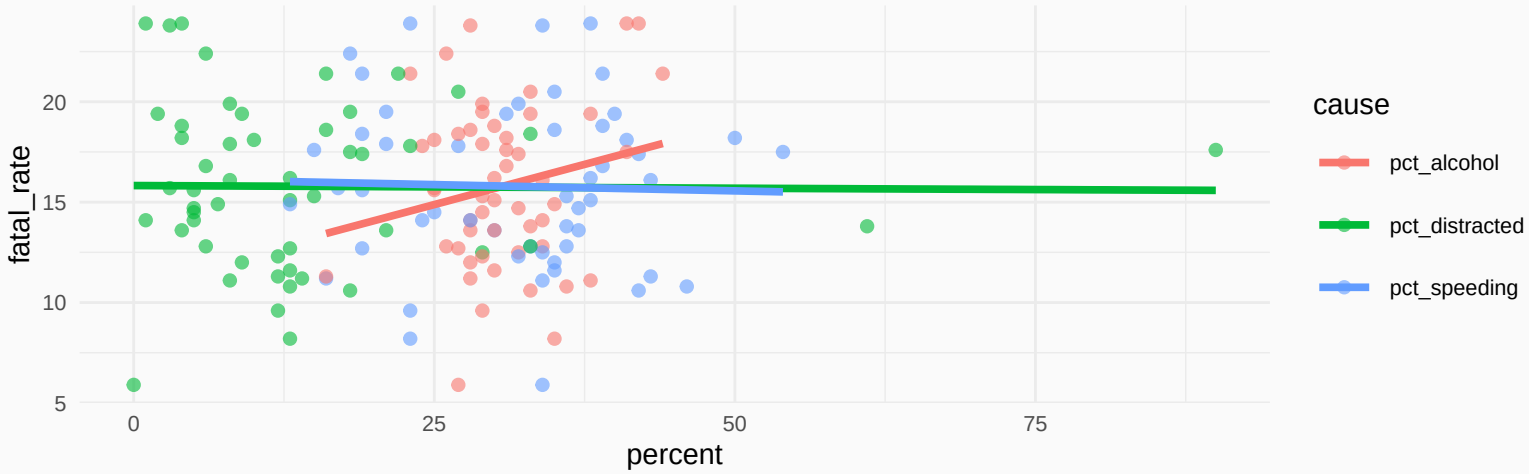
```
drivers_long <- drivers_clean |>
  mutate(pct_distracted = 100 - pct_not_distracted) |>
  select(state, fatal_rate, pct_speeding, pct_alcohol,
         pct_distracted) |>
  pivot_longer(
    cols = starts_with("pct"),
    names_to = "cause",
    values_to = "percent"
  )
drivers_long
```

```
## # A tibble: 153 x 4
##   state   fatal_rate cause          percent
##   <chr>        <dbl> <chr>            <dbl>
## 1 Alabama       18.8 pct_speeding        39
## 2 Alabama       18.8 pct_alcohol         30
## 3 Alabama       18.8 pct_distracted       4
## 4 Alaska        18.1 pct_speeding        41
## # i 149 more rows
```

Wide to long in one picture

- Before: one row per state, with one column per cause.
- After: one row per state **and** cause, with columns `cause` and `percent`.
- Now `cause` can be mapped to an aesthetic such as `color`.

```
ggplot(drivers_long,
       aes(x = percent, y = fatal_rate, color = cause)) +
  geom_point(alpha = 0.6, size = 1.2) +
  geom_smooth(method = "lm", se = FALSE)
```



Wrap-up

Common mistakes

Symptom	Likely cause
`+` at the start of a line	The `+` must end the previous line
Points are the wrong color and a legend appears	A constant inside `aes()`
A pipeline's result "disappears"	It was not assigned with `<-` or printed
A group called `NA` in a summary	A recode or join missed some rows
A bar chart that shows the wrong heights	`geom_bar()` where `geom_col()` was needed
Cannot map a variable to `color`	The variable is spread across columns

Questions you should be able to answer

1. What is the difference between `geom_point(aes(color = x))` and `geom_point(color = "red")`?
2. In a pipeline, what does `|>` do?
3. You want to color points by `cause`, but the causes are in three different columns. What should you do first?

Read Textbook chapters

R for data science (2nd edition)

Chapter 1: ggplot

Chapter 3: Data transformation

Chapter 5: Tidy data

Chapter 19: Joins

and practice exercises in the textbook

https://r4ds.hadley.nz/data-tidy.html